Aterrizaje lunar con Deep Q-Networks: implementación, análisis de la política y estudio de ablación

Thiago Rivas
C.I. 5.475.680-3
Facultad de Ingeniería
Universidad de la República
Montevideo, Uruguay
thiago.rivas0021@gmail.com

Juan Duarte
C.I. 5.201.302-9
Facultad de Ingeniería
Universidad de la República
Montevideo, Uruguay
juan.duarte@fing.edu.uy

Agustin Occhiuzzi
C.I. 5.480.562-4
Facultad de Ingeniería
Universidad de la República
Montevideo, Uruguay
agustin.occhiuzzi@fing.edu.uy

***Resumen*. Se implementó un agente DQN para LunarLander-v3 y se estudió cómo aprende, qué política obtiene y qué pasa al quitarle tres partes al algoritmo. En la calibración se comparó una configuración de referencia con nueve alternativas, y solo se cambió la duración de la exploración. El agente principal, entrenado durante 1200 episodios, logró un retorno de al menos 200 en el 81 % de 200 escenarios de test (30000 a 30199); en la evaluación de la plantilla (10000 a 10199), lo logró en el 77,5 %, frente al 0 % del agente aleatorio. Los cortes de la política muestran que el agente aprendió a frenar la caída y a corregir el giro. El estudio de ablación tiene 32 agentes, ocho por condición, entrenados durante 300 episodios cada uno en la misma computadora. Para comparar las variantes se usó el test exacto bilateral de signos emparejado con la corrección de Holm: el agente completo tuvo mejor promedio que las tres variantes, pero ningún contraste fue significativo. Estos resultados tienen límites: se usaron pocas semillas y entrenamientos cortos en la ablación.**

***Palabras clave*. aprendizaje por refuerzos, DQN, LunarLander, ablación**

I. Introducción

En LunarLander-v3 hay que aterrizar una nave usando un motor principal y dos motores laterales. Se usó la versión discreta de Gymnasium [1], sin viento. El estado es $s = (x, y, v_x, v_y, \theta, \omega, c_1, c_2)$: posición, velocidad, ángulo, velocidad angular y dos indicadores que dicen si cada pata toca el suelo. Hay cuatro acciones: no hacer nada, encender el motor izquierdo, encender el principal o encender el derecho.

El objetivo fue entrenar un agente con DQN [2] y ver qué aportan la memoria de repetición, la red objetivo y la exploración. Se usó el criterio de la tarea: un episodio es exitoso si su retorno total $R = \sum_t r_t$ es de al menos 200, y la tasa de éxito tiene que ser mayor que 10 %. Este criterio mira solo la recompensa, así que no comprueba directamente que la nave haya aterrizado.

II. Solución DQN implementada

La clase `AgenteDQN` usa una red neuronal Q_θ que, dado un estado, estima cuánto vale cada acción. La red recibe los ocho valores del estado, tiene dos capas ocultas con ReLU y devuelve cuatro números, uno por acción. Cada número estima el retorno descontado que se espera obtener si se toma esa acción [3]. Si desde el paso t se reciben las recompensas $r_t, r_{t+1}, \ldots$, ese retorno es $G_t = \sum_{k\geq 0} \gamma^k r_{t+k}$, o sea que cada recompensa se multiplica por γ una vez por cada paso que falta para recibirla. Con $\gamma = 0{,}99$, una recompensa que llega 100 pasos después todavía vale cerca del 37 % ($0{,}99^{100} \approx 0{,}37$). Por eso el agente tiene en cuenta cosas que pasan bastante más adelante, como el premio por aterrizar. La red tiene 4996 parámetros. El DQN original trabajaba con imágenes de juegos de Atari [2]. Acá, en cambio, la red recibe directamente el vector de estado y usa solo capas completamente conectadas. No se normalizan las observaciones ni se recortan las recompensas.

Cada transición (s, a, r, s', d) se guarda en una memoria de repetición (*replay*) con capacidad fija. Cada vez que se actualiza la red, se sacan al azar 64 transiciones distintas de esa memoria (un minilote). Así se aprovecha cada experiencia más de una vez y se evita aprender solo de transiciones seguidas. Esto importa porque de un paso al siguiente la nave está casi en el mismo lugar. Si la red aprendiera de las transiciones en orden, muchas actualizaciones seguidas la empujarían hacia el mismo lado: se adaptaría a la situación del momento y podría olvidar lo que aprendió antes.

Además se usa una segunda red, la red objetivo Q_{θ^-}. Empieza siendo una copia de Q_θ, no se entrena y cada cierto número de pasos se vuelve a copiar. Con ella se calcula el valor objetivo de cada transición:

$$z = r + \gamma(1 - d) \max_{a'} Q_{\theta^-}(s', a'). \tag{1}$$

En z, r es la recompensa que realmente se recibió. La parte $\max_{a'} Q_{\theta^-}(s', a')$ es lo que la red estima que se puede ganar desde s' eligiendo la mejor acción, y γ lo descuenta. El factor $(1 - d)$ vale 0 cuando el episodio terminó. En ese caso el objetivo es solo r, porque después del final ya no hay más recompensas.

Si el objetivo se calculara con la misma red que se está entrenando, cada vez que la red cambia también cambiaría el

Tabla I
CONFIGURACIÓN FINAL

Parámetro	Valor
Red Q	$8 \to 64 \to 64 \to 4$, ReLU
Adam	$\eta = 10^{-3}$, $\beta_1 = 0{,}9$, $\beta_2 = 0{,}999$
Descuento	$\gamma = 0{,}99$
Memoria / minilote	50 000 / 64 transiciones
Inicio / frecuencia	1000 pasos / cada 4 pasos
Copia a red objetivo	Cada 1000 pasos de entorno
Exploración	Lineal, $1 \to 0{,}05$ en 100 episodios
Pérdida / gradiente	Huber ($\beta = 1$) / norma ≤ 10
Episodios (hasta 1000 pasos)	1200 principal, 300 estudio

valor z al que se quiere llegar, como un perro que se persigue la cola. Con la red objetivo fija entre copias, el objetivo se queda quieto un tiempo y el entrenamiento es más estable.

La red se entrena para que su predicción $Q_\theta(s, a)$ se acerque a z, usando la pérdida de Huber (SmoothL1 en PyTorch) promediada sobre el minilote. El valor z se toma como un número fijo, sin calcular gradientes a través de él. Los pesos se ajustan con Adam, y se pone un límite al tamaño del gradiente para que ninguna actualización sea demasiado grande.

La variable d vale 1 solo cuando el episodio termina de verdad (`terminated`). Si el episodio se corta porque llegó al límite de 1000 pasos (truncamiento), se sigue usando el valor futuro, porque la nave no llegó a un final real. Si ese corte se tratara como un final, la red aprendería que en esos estados no queda nada por ganar, aunque la nave podría haber seguido volando y aterrizado. Además, como el estado no incluye el tiempo, la red no podría distinguir esos estados de otros iguales que no llegaron al límite.

Durante el entrenamiento se usa la estrategia ε-greedy: con probabilidad ε se elige una acción al azar y, si no, la de mayor valor. Después de e episodios, ε vale

$$\varepsilon_e = \max\left(0{,}05,\ 1 - 0{,}95\frac{e}{100}\right). \tag{2}$$

Al principio la red todavía no aprendió nada y sus valores son casi al azar, así que conviene probar muchas acciones para descubrir cuáles dan buenas recompensas. A medida que aprende, conviene usar más lo que ya sabe. Desde el episodio 100 queda un 5 % de acciones al azar, para seguir probando un poco.

En la evaluación siempre se elige la acción de mayor valor, $\arg\max_a Q_\theta(s, a)$, sin explorar y sin cambiar los pesos.

La tabla I resume la configuración final. Siempre se entrenan todos los episodios y se evalúan los pesos del final, sin quedarse con el mejor momento del entrenamiento. El agente principal se entrena 1200 episodios, como en la plantilla. La calibración y la ablación usan 300 episodios por agente, porque la consigna recomienda usar menos episodios en la ablación, que implica muchos entrenamientos.

Las semillas de entrenamiento son 42, 1042, 2042, 3042, 4042, 5042, 6042 y 7042. Se fijan las semillas de Python, NumPy, PyTorch y del espacio de acciones. El episodio i (contando desde cero) empieza con la semilla de entrenamiento más i. Para no evaluar en los mismos escenarios que se usaron para decidir, la calibración se evaluó en 100 escenarios de validación (50000 a 50099) y los resultados finales, en 200 escenarios de test (30000 a 30199). Todas las corridas se hicieron en la misma computadora. Fijar las semillas permite repetir exactamente un entrenamiento en esa máquina. En otra computadora, pequeñas diferencias en los cálculos hacen que el entrenamiento se separe después de algunos episodios y los números cambian.

Tabla II
CALIBRACIÓN: RETORNO MEDIO EN 100 ESCENARIOS DE VALIDACIÓN

Configuración	s42	s1042	s2042	Media	Gana
A0 (referencia)	−23,4	150,6	213,3	113,5	–
A1: 128×128, $\eta = 5 \cdot 10^{-4}$	85,6	12,8	16,7	38,4	1/3
A2: $\eta = 5 \cdot 10^{-4}$	0,5	−14,4	−49,5	−21,1	1/3
A3: $\eta = 5 \cdot 10^{-3}$	237,3	156,9	15,4	136,5	2/3
Copia cada 250	222,8	116,6	−8,6	110,3	1/3
Copia cada 4000	54,5	−17,3	4,8	14,0	1/3
ε en 100 episodios	−14,7	231,1	256,5	157,7	3/3
ε en 400 episodios	−147,9	−48,2	−49,7	−81,9	0/3
Memoria 10 000	123,8	242,4	93,4	153,2	2/3
Memoria 150 000	−8,0	−10,2	202,7	61,5	1/3

Cada alternativa cambia solo lo indicado respecto de A0. Gana: semillas en las que supera a A0.

III. SELECCIÓN DE HIPERPARÁMETROS

Se partió de una configuración de referencia A0, igual a la de la tabla I pero con la exploración en 200 episodios, y se la comparó con alternativas que cambian una sola cosa: la red y la tasa de aprendizaje, cada cuántos pasos se copia la red objetivo, en cuántos episodios baja ε y el tamaño de la memoria. Para la copia, la exploración y la memoria se probó un valor más chico y otro más grande que el de A0. A1 cambia a la vez el tamaño de las capas y la tasa respecto de A0. Por eso se agregó A2: entre A0 y A2 cambia solo la tasa, y entre A2 y A1 cambia solo el tamaño. A3 es igual a A0 pero con una tasa más alta. Cada configuración se entrenó 300 episodios con las semillas 42, 1042 y 2042 y se evaluó en los escenarios de validación (tabla II).

La regla fue cambiar un valor de A0 solo si la alternativa lo supera en las tres semillas, porque con tres semillas y lo mucho que varía DQN, tener mejor promedio no alcanza. Por ejemplo, A3 y la memoria de 10 000 tienen mejor promedio que A0, pero quedan por debajo en una semilla. Solo bajar ε en 100 episodios superó a A0 en las tres, y se adoptó. Igual, esta regla puede cumplirse por azar: si una alternativa fuera igual a A0, lo superaría en las tres semillas con probabilidad de alrededor de 1/8, y con nueve alternativas es esperable que alguna lo logre por suerte. Lo que más respalda la elección es que las tres duraciones quedan en un orden coherente: 400 episodios da el peor resultado, 200 queda en el medio y 100 el mejor. Tiene sentido: con 300 episodios, explorar durante 200 deja pocos episodios para usar lo aprendido, y explorando durante 400, al terminar ε todavía vale cerca de 0,3. Además, en la semilla 42 la ventaja fue de solo 8,7 puntos, así que con tres semillas no se puede decir que sea la mejor opción en general.

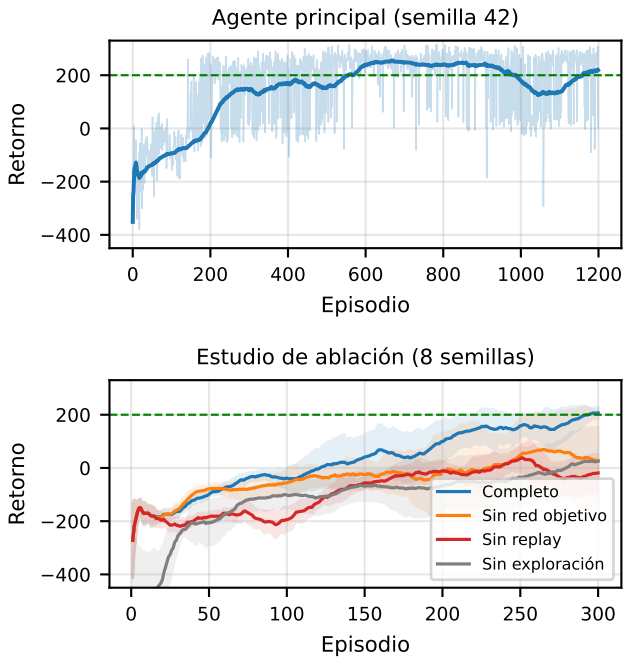


Figura 1. Retorno durante el entrenamiento. Arriba: agente principal, por episodio (claro) y promedio de los últimos 100 episodios (oscuro). Abajo: estudio de ablación, promedio de los últimos 20 episodios, promediado sobre ocho semillas. Las bandas muestran ± 1 DE entre semillas y no son intervalos de confianza. El eje vertical se cortó en -450. La línea discontinua es el umbral de éxito.

IV. Aprendizaje y desempeño final

El agente principal (configuración final, semilla 42) se entrenó 1200 episodios. La figura 1 (arriba) muestra que mejora: el promedio de los últimos 100 episodios pasa de -95 en el episodio 100 a 145 en el 300 y llega a 255 en el 668. Después baja a 125 en el episodio 1045 y vuelve a subir hasta 219 al final. DQN no mejora siempre: la red se sigue entrenando con datos que ella misma genera y puede empeorar por un tiempo. Por eso se evalúan los pesos del final y no los del mejor momento.

En los 200 escenarios de test, el agente principal obtuvo un retorno medio de 240,2 y 162 éxitos (81 %). Terminó posado en 165 episodios, chocó en 3 y en 32 llegó al límite de pasos. En la evaluación de la plantilla (escenarios 10000 a 10199) obtuvo $235{,}79 \pm 67{,}42$ de retorno y 155 éxitos de 200 (77,5 %). El agente aleatorio, con su generador fijado en 42 y en los mismos escenarios, obtuvo $-183{,}17 \pm 111{,}96$ y ningún éxito. Acá $\pm$ es la desviación estándar de los 200 retornos.

V. Análisis de la política

Para ver qué aprendió el agente principal se hicieron tres cortes. En cada uno se parte de un estado de referencia y se mueven solo dos de sus valores en una grilla de 101×101: (x, v_x), (θ, ω) o (y, v_y). Los otros seis quedan fijos. Esto hace falta porque la política depende de ocho variables y no se puede dibujar entera. Fijando seis, la acción depende solo de dos y se puede mostrar como una imagen, aunque cada corte muestra solo una parte de la política. El estado de referencia tiene $x = v_x = \theta = \omega = c_1 = c_2 = 0$, $y = 0{,}8$ y $v_y = -0{,}3$. La figura 2 muestra, en cada punto, la acción que elige la red.

En el corte vertical aparece una frontera inclinada: el motor principal se enciende cuando la nave cae rápido, y cerca del suelo también cuando cae más despacio. Esto tiene sentido, porque el motor no puede frenar la nave de golpe: cuanto más rápido cae, más altura necesita para frenar. Por eso, si cae rápido conviene empezar a frenar antes, y si cae despacio alcanza con frenar cerca del suelo. Más arriba de esa zona aparecen tanto no hacer nada como el motor derecho, así que el agente no sigue simplemente la regla de frenar o no frenar.

En el corte del ángulo, a cada lado de una frontera que depende de θ y ω se usa un motor lateral distinto. Cerca de esa frontera, el motor derecho reduce el ángulo y el izquierdo lo aumenta. Parece que el agente corrige a la vez la inclinación y el giro, aunque esto no alcanza para decir que siga una regla lineal en todos los estados. En los cortes de los ocho agentes completos del estudio, guardados en el notebook, se ve algo parecido, aunque cambian las fronteras y las zonas donde no hace nada o usa el motor principal.

En el corte horizontal, si la nave se mueve hacia la izquierda ($v_x < 0$), el agente usa casi siempre el motor derecho. Si se mueve hacia la derecha a velocidad moderada ($0{,}1 < v_x < 0{,}9$) y ya está a la derecha de la plataforma ($x > 0$), usa el izquierdo. Las dos cosas parecen servir para frenar ese desplazamiento. Con velocidades hacia la derecha más altas no hace nada. La división depende también de x y no es simétrica. Con un solo estado de referencia no se puede decir que el agente prefiera un lado en general.

VI. Estudio de ablación

VI-A. *Variantes y test estadístico*

En cada condición se quita una sola parte. Sin red objetivo, el valor futuro se calcula con la misma red que se está entrenando, y z se sigue tomando como un número fijo. Sin replay, cada actualización usa solo la transición que acaba de pasar. Como se mantienen el inicio en el paso 1000 y la actualización cada cuatro pasos, las otras tres transiciones no se usan para aprender. Sin exploración, $\varepsilon = 0$ desde el principio, aunque sigue habiendo azar en los pesos iniciales y en el entorno.

Por lo visto en la sección II, sin red objetivo se espera un entrenamiento más inestable, porque el valor al que se quiere llegar cambia en cada actualización. Sin replay se espera que la red aprenda de datos muy parecidos entre sí, use cada transición como mucho una vez y se olvide de lo que aprendió. Sin exploración se espera que el agente repita lo que su red inicial cree mejor y tarde más en descubrir acciones útiles, aunque el azar del entorno y de los pesos le da algo de variedad.

Las cuatro condiciones (completo y tres variantes) se entrenaron 300 episodios con las mismas ocho semillas, en la misma computadora, y se evaluaron en los 200 escenarios de test. Cada agente entrenado cuenta como un solo dato. Sus

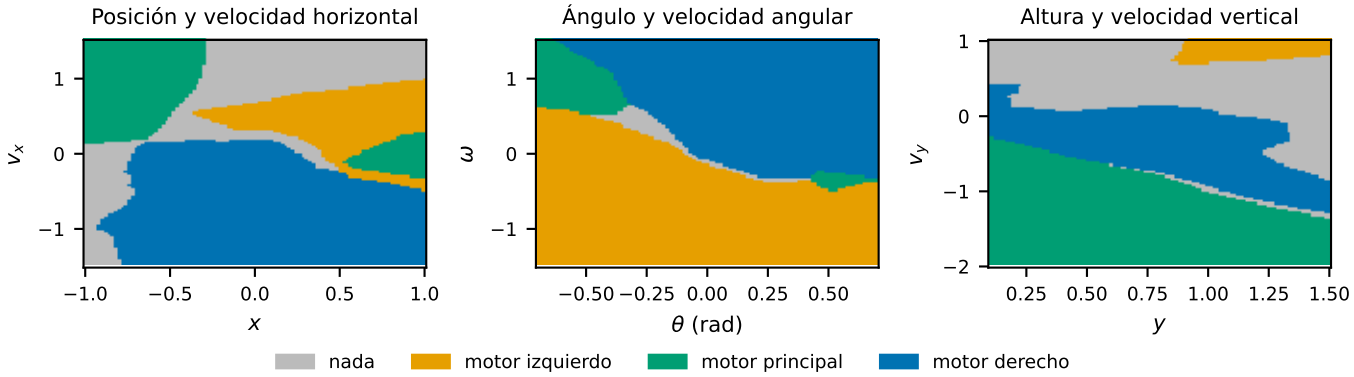


Figura 2. Agente principal: acción que elige la red en cada punto. En cada panel solo cambian las dos variables de los ejes. Las demás quedan como en el estado de referencia.

Tabla III
EVALUACIÓN DEL ESTUDIO EN LOS 200 ESCENARIOS DE TEST

Semilla	Completo	Sin objetivo	Sin replay	Sin explor.
42	−2,1	−166,9	−148,0	−165,5
1042	231,5	150,8	−76,8	63,4
2042	250,5	205,8	17,3	156,3
3042	72,3	73,8	−39,7	−72,1
4042	−23,8	178,1	118,3	−133,8
5042	38,2	219,8	15,0	69,4
6042	220,8	−166,5	−3,3	127,1
7042	194,5	−524,6	176,3	170,9
Media	122,7	−3,7	7,4	27,0
DE entre semillas	113,2	262,2	103,2	132,7
Éxitos / 1600	710	563	148	284
Éxito (%)	44,38	35,19	9,25	17,75
Posado	822	727	354	414
Choque o salida	55	657	1218	410
Truncado	723	216	28	776

Las tres últimas filas cuentan cómo terminan los 1600 episodios de cada condición.

200 episodios de evaluación sirven para medir qué tan bueno es ese agente, pero no son 200 entrenamientos distintos. Todos vienen del mismo entrenamiento: si una semilla dio una red mala, sus 200 episodios van a ser malos juntos. Como lo que se compara es la forma de entrenar, contarlos como 200 datos haría que las diferencias parecieran mucho más seguras de lo que son. En la tabla III, cada celda de las primeras filas es el promedio de esos 200 retornos.

Para comparar cada variante con el completo se emparejaron los agentes por semilla de entrenamiento. Para cada semilla s se calculó $D_s = \bar{R}_{\text{completo},s} - \bar{R}_{\text{variante},s}$, donde cada $\bar{R}$ es el promedio de sus 200 retornos de test. Se usó el test exacto bilateral de signos, implementado con `scipy.stats.binomtest`. Se cuentan las diferencias positivas (k) entre las no nulas (n); los empates exactos se excluyen antes de calcular el test. En estos datos no hubo empates.

La hipótesis nula es $P(D_s > 0 \mid D_s \neq 0) = 0{,}5$: una diferencia no nula tiene igual probabilidad de favorecer a cualquiera de las dos condiciones. Bajo esa hipótesis, k sigue una distribución binomial con n ensayos y probabilidad $0{,}5$. El valor p bilateral mide cuán extremo es el conteo observado bajo ese modelo. El test requiere independencia entre pares, pero permite dependencia dentro de cada par y no exige normalidad. Solo usa los signos: no contrasta la igualdad de medias ni tiene en cuenta el tamaño de las diferencias. Por eso se informa también su media, Δ, como descripción de magnitud.

Tabla IV
TEST EXACTO BILATERAL DE SIGNOS CONTRA EL COMPLETO (OCHO PARES POR COMPARACIÓN)

Ablación	Δ	k/n	p signos	p Holm
Sin objetivo	126,4	5/8	0,7265625	0,7265625
Sin replay	115,3	7/8	0,0703125	0,2109375
Sin explor.	95,8	7/8	0,0703125	0,2109375

Δ: media de las diferencias emparejadas (completo menos variante), descriptiva. k/n: diferencias positivas/no nulas. Sin empates; los ceros se excluyen de n. Holm: tres comparaciones, $\alpha = 0{,}05$.

Como hay tres comparaciones, se aplica la corrección de Holm [4], con $\alpha = 0{,}05$ para el conjunto. Se rechaza cada hipótesis si su p ajustado es menor o igual a α. Bajo los supuestos del test, esta corrección limita a 0,05 la probabilidad de al menos un rechazo falso. La reutilización de tres semillas de calibración, discutida en las limitaciones, hace que el análisis sea exploratorio.

VI-B. Resultados

La tabla IV muestra que el agente completo tiene mejor promedio que las tres variantes. Supera a la variante sin objetivo en cinco de ocho pares, y a las variantes sin replay y sin exploración en siete de ocho. Estas dos últimas tienen el mismo p sin corregir, $0{,}0703125$, y ajustado, $0{,}2109375$; para la variante sin objetivo, ambos valores son $0{,}7265625$. Ningún contraste rechaza la hipótesis nula tras Holm. Esto no demuestra equivalencia entre condiciones ni que los componentes no sirvan. Las diferencias medias de 95,8 a 126,4 puntos son descriptivas: el test solo contrasta el signo de las diferencias, no su magnitud.

Los finales de los episodios (tabla III) muestran que cada variante falla de una forma distinta. Sin replay, 1218 de sus 1600 episodios terminan en choque o salida, contra 55 del completo. Cada actualización depende de una transición muy parecida a las anteriores, así que los ajustes de la red son muy irregulares y la red se adapta a lo último que vio. Esto sugiere que no llega a aprender bien a frenar antes de tocar el suelo. Sin exploración, la diferencia con el completo está en los choques (410 contra 55) y en los posados (414 contra 822). Los truncados son parecidos en las dos (776 y 723), así que no la distinguen. Sin red objetivo, el resultado cambia mucho según la semilla: en algunas iguala o supera al completo y en otras obtiene resultados muy malos (semilla 7042: $-524{,}6$). Por eso su DE es más del doble que la del completo, como se esperaba sin red objetivo.

Estos finales describen el comportamiento, pero no son tasas de éxito. Por ejemplo, el agente completo tiene 822 finales posados, pero solo 710 episodios con $R \geq 200$. Una nave que aterriza gastando mucho combustible puede no llegar a 200.

VII. Limitaciones

Todas las corridas del estudio tienen los mismos 300 episodios, pero no la misma cantidad de pasos ni de actualizaciones: usaron entre 32 968 y 241 521 pasos, según cuánto duraron sus episodios. Además, con 300 episodios los agentes todavía están aprendiendo. Con la semilla 42, el agente completo de la ablación es el mismo entrenamiento que los primeros 300 episodios del agente principal. A los 300 episodios obtiene $-2{,}1$ en test, y el principal, después de 1200, 240,2. Que el promedio de entrenamiento de los episodios 201 a 300 sea 145 no lo contradice: ese promedio mezcla muchas redes distintas, porque la red cambia cada cuatro pasos, y la red exacta del episodio 300 llega al límite de pasos en 189 de los 200 episodios de test. Por eso las diferencias de la ablación podrían cambiar con entrenamientos más largos. Además, el 81 % del agente principal sale de un solo entrenamiento, con una semilla que a los 300 episodios está entre las peores del completo, así que con otra semilla el resultado podría ser distinto.

Ocho semillas son pocas para saber bien cuánto varían los resultados, y el test de signos pierde información al ignorar la magnitud de las diferencias. La calibración usó solo tres semillas. Esas tres son también las primeras semillas de la ablación: con las semillas 42, 1042 y 2042, los agentes completos de la ablación son los mismos entrenamientos con los que se eligió la configuración, lo que puede favorecerlos un poco. Todo se corrió en una sola computadora, y en otra los números cambiarían. Por último, los cortes muestran solo tres planos, y la conclusión sobre el replay vale para esta forma de quitarlo.

VIII. Conclusiones

Se implementó un agente DQN que cumple el criterio de éxito de la tarea: entrenado 1200 episodios, logra un 81 % de éxito en escenarios de test (30000 a 30199); en la evaluación de la plantilla (10000 a 10199), logra un 77,5 % frente al 0 % del agente aleatorio. En los cortes se ve que aprendió a frenar y a corregir el ángulo, con algunas asimetrías y diferencias entre semillas.

En la ablación, quitar cualquiera de las tres partes empeora el promedio, y cada variante falla de una forma que tiene sentido con lo que hace esa parte. Pero con ocho pares el test de signos no rechaza ninguna hipótesis después de corregir las tres comparaciones. Esto no demuestra equivalencia ni permite afirmar que la mejora se generalice a otras semillas. Para estudiar esa generalización haría falta usar nuevas semillas, separadas de la calibración, y entrenamientos más largos.

Uso de Inteligencia Artificial

Se usaron herramientas de IA para desarrollar y revisar el código, verificar los resultados, y para la corrección gramática de este informe. Todas las cifras se contrastaron con los datos guardados de las corridas.

Referencias

[1] M. Towers *et al.*, "Gymnasium: A standard interface for reinforcement learning environments," arXiv:2407.17032, 2024.

[2] V. Mnih *et al.*, "Human-level control through deep reinforcement learning," *Nature*, vol. 518, pp. 529–533, 2015, doi: 10.1038/nature14236.

[3] R. S. Sutton y A. G. Barto, *Reinforcement Learning: An Introduction*, 2.ª ed. MIT Press, 2018.

[4] S. Holm, "A simple sequentially rejective multiple test procedure," *Scand. J. Statist.*, vol. 6, no. 2, pp. 65–70, 1979.